Campus Supply Desk

COMP3415 Assignment 2 - local review report

Prepared November 8, 2026. This report covers post-Checkpoint-A testing, refactoring, defect correction and CR-01 express delivery. Work is local on branch `assignment-a2/local-improvements`. No push or D2L submission has occurred.

Result

The original four tests pass, and the expanded suite contains 49 passing test methods. Three demonstrated baseline defects are corrected: student discount eligibility at exactly $50, duplicate-line overselling, and stock inflation after repeated cancellation. Quote and placement now share one pricing and validation path. Express delivery is supported through all three interfaces at an unconditional 1,200-cent fee.

Seven targeted, deliberately incorrect mutations are all detected by assertion failures. Recorded evidence preserves the failing baseline, the structural refactor before corrections, the fixes, the express tests before implementation, and the final passing suite. These results support the cases exercised, not a claim of exhaustive correctness.

Basis and submission status

The user confirmed that Checkpoint A had already been completed and submitted. Its contents were not available and have not been recreated or described as AI-independent work. This is an AI-assisted post-checkpoint report; the student must review its analysis and conclusions before submission.

The complete Assignment 2 handout was not supplied. Therefore this review copy cannot certify compliance with unprovided report formatting, grading criteria, minimum test counts or required AI transcript formats. The separate supplied CR-01 was available and is implemented. The repository link for the student's own hosted repository remains pending the user's explicit push approval.

Sources and provenance

- Instructor starter: https://github.com/iiConick/COMP3415-A2-Campus-Supply-Desk
- Baseline commit: `82ed75e0714e50c086a13ee491f51b349814d8a4`.
- Original `FUNCTIONAL_SPEC.md`: behavior oracle, rather than starter output.
- `docs/CR-01.md`: local copy of the supplied express-delivery request.
- `evidence/`: actual automated test output and verification metadata.
- `docs/ai-review.md`: recommendations, choices and limitations.

1. System understanding

Campus Supply Desk is an offline Python terminal application for a small campus merchandise catalog. It supports quotes, order placement, history, lookup and cancellation. Prices are integer Canadian cents. It has no payment integration, authentication, taxes, addresses or shipping workflow.

Components and execution flow

`supplydesk/__main__.py` invokes `cli.main`. The argument parser chooses an individual command or, when none is specified, launches `menu.run_menu`. Both adapters delegate business behavior to `OrderService`. The menu converts quantity text to integers and formats cents as dollars; individual commands return JSON with integer cents and use nonzero exit statuses for rejected input.

`OrderService` loads state from `JsonStore`, validates requests against current stock, computes line totals and prices, and returns structured dictionaries. Quote operations do not save or reserve stock. Placement validates the entire request, creates the next sequential ID, reduces stock, appends a snapshot and saves. Cancellation restores units once, preserves the original prices and keeps the order in history with status `cancelled`.

`storage.py` reads and writes schema-version-1 JSON. When a save is absent, `catalog.json` supplies five initial products and stock. An existing malformed or unreadable file propagates an error instead of resetting history. Normal data uses `.supplydesk/state.json`; tests isolate every service with temporary files. Data persists across newly constructed service instances and processes.

Rules used as test oracles

- Customer is exactly regular or student; quantity is an integer 1-99, excluding booleans, fractions and numeric strings at the service boundary.
- Complete requests must fit stock, including duplicate product lines.
- Students receive 10% off merchandise at a subtotal of at least 5,000 cents; half cents round upward using integer arithmetic.
- Pickup is free; standard delivery costs 600 cents and is free at 10,000 cents after merchandise discount; express always costs 1,200 cents.
- Invalid requests do not consume IDs or alter successful saved operations.
- IDs are monotonic; cancellation preserves history and is idempotent.

Scope boundaries retained

JSON storage and public result shapes remain. No database, external service, payment system, concurrent-writer support, crash-recovery guarantee or schema migration was introduced. The current functional spec includes clearly marked CR-01 additions; the original remains accessible at the baseline commit.

2. Code problems and supporting evidence

All original line references below refer to `supplydesk/service.py` at baseline commit `82ed75e`. This post-checkpoint review does not claim these are the student's independently submitted five problems.

P1. Duplicated validation and pricing - maintainability

Original quote lines 15-60 and placement lines 65-112 repeat validation, catalog lookup, item construction, discount and delivery rules. A rule change must be implemented twice and can create divergent quote/order prices. Extracting `_build_quote` gives both operations one implementation, while placement still loads fresh state and performs its own validation before saving.

P2. Discount excludes the exact threshold - correctness

Original lines 48 and 98 use `subtotal > 5000`, contrary to the specified inclusive threshold. Two bags cost exactly 5,000 cents: the starter returns no discount instead of 500 cents. The pricing regression fails for both quote and placement before correction. The final comparison is `>=`.

P3. Stock validation ignores combined duplicate quantities - correctness

Original lines 36 and 86 validate each item in isolation. Notebook lines of 11 and 10 both fit a stock of 20 individually, then placement can save stock of -1. The final builder tracks requested quantity per validated product ID before any inventory mutation. Rejection preserves prior save bytes and IDs; a valid 9 + 11 request retains its two lines and cancels correctly.

P4. Repeated cancellation inflates stock - correctness

Original cancellation lines 142-147 always restore ordered units, even when the order is already cancelled. Cancelling two mugs twice produces stock 10 instead of the initial 8. The final method returns an existing cancelled order before changing inventory or writing. The regression checks reopening, returned snapshot, stock and byte-for-byte save stability.

P5. Repeated search and inventory loops - maintainability

Original lookup lines 128-131 and cancellation lines 135-141 implement the same search and error policy. Placement lines 114-117 and cancellation lines 142-145 repeat nested stock loops with opposite signs. `_find_order` and `_change_stock` centralize these operations. The stock helper indexes trusted internal lines; public product validation keeps the simple scan to reject unhashable IDs with ValueError. This is a clarity improvement, not a measured speed claim.

P6. Repeated business-rule literals - maintainability

Original lines 48-52 and 98-102 embed 5,000, 10%, 10,000 and 600 in both pricing paths. Named constants now document their monetary roles and keep each rule in one place. CR-01 adds a separate `EXPRESS_DELIVERY_CENTS = 1200` constant.

3. Refactoring and preservation of behavior

Separate structure from corrections

The shared quote builder was extracted before changing the three defective rules. At that stage, all 27 baseline tests produced exactly the same six failing assertions as before refactoring. This provides evidence that the structural change retained the exercised existing behavior. A subsequent commit changed the inclusive discount boundary, combined stock check and repeat-cancellation guard, after which all 27 tests passed.

Local commit	Purpose	Observed validation
a2a89fc	Capture starter baseline and requirements tests	Starter 4 pass; expanded 27 expose 6 failures
85b9ab3	Extract shared quote, lookup and stock operations	Same 27 tests and same 6 failures
0637165	Correct three demonstrated defects	27 pass
3215404	Add express and adapter acceptance tests	49 tests; expected pre-feature failures/errors
e04cbd5	Implement CR-01 and documentation	49 pass

Final design

`quote()` loads state and passes it to `_build_quote`. `place_order()` also loads state and invokes that builder, then records the returned pricing and item snapshot alongside the new ID and status. It adjusts inventory only after the builder returns successfully. A quote remains advisory and does not reserve stock; a later placement rechecks current availability.

`_find_order` supplies one unknown-ID policy for retrieval and cancellation. `_change_stock` adjusts validated lines in either direction. A cancellation guard prevents a second restoration. No saved schema migration was needed: fulfillment was already stored as a string in each order snapshot.

Preservation checks

Tests cover regular/pickup defaults, JSON output, integer monetary types, menu confirmation/decline paths, quote non-mutation, sequential IDs after rejection and cancellation, history ordering, current stock after reopening, and original prices after catalog changes. Valid duplicate lines remain separate. Existing malformed data reports an error rather than being silently replaced.

Intentional behavioral changes are limited to the three demonstrated fixes and express-delivery support. Whole-program equivalence is not claimed; only specified behavior and recorded cases are supported by the evidence.

4. Testing strategy and results

Risk-based selection

The suite prioritizes money boundaries, inventory integrity, invalid-request atomicity and persisted order history. Fixed expected amounts come from the functional rules and CR-01 examples, rather than from production helpers. Equivalence partitions cover supported/unsupported customers and fulfillment, valid/invalid products, empty/malformed requests, and quantity type/range.

Boundary cases include 1 and 99 quantities, 0 and 100 rejections, stock exactly available and insufficient, a $50 merchandise subtotal, and delivery values below/at/above $100 after discount. A synthetic 5,005-cent catalog item tests half-up rounding because normal catalog prices cannot reach a half-cent discount. A fixture with stock 100 makes quantity 99 testable independently of bundled stock limits.

Test layers

- Service tests cover validation, pricing, snapshots and lifecycle with real JSON persistence in temporary directories.
- Command tests spawn `python -m supplydesk`, parse JSON, and inspect real exit statuses and errors. They cover quote, place, history, show and cancellation.
- Menu tests script input and capture displayed output, including defaults, invalid quantity retry, confirmation, express placement and clean exits.
- Permission failure and EOF/Ctrl+C are simulated; these are not claims of physical device or operating-system permission testing.

Observed outcomes

The starter's 4 tests pass. Adding baseline requirements tests produces 27 methods with 6 failing assertions across 4 methods/subtests. The structural refactor retains those results; corrections make all 27 pass. Adding CR-01 and adapter tests yields 49 methods with 5 failures and 8 errors before express implementation. The completed code passes all 49, including the original four. Subtests exercise more input combinations than the method count indicates.

Seven hand-selected mutations are detected: exclusive discount threshold, per-line duplicate stock check, repeated stock restoration, waived express, discounted express, downward half-cent rounding, and removal of the CLI express choice. Isolated copies ensure the verification does not damage working source.

Evidence and limits

`docs/testing-evidence.md` maps cases to tests and logs. Final run output is in `evidence/07-final-tests.txt`; mutation outputs are in `08-targeted-mutations.json`; `09-verification-metadata.json` records Python version, UTC time, command and source/test hashes. Windows Python 3.12.10 was tested. No code-coverage percentage or exhaustive mutation score is claimed. Python 3.10 was not separately tested.

5. CR-01 acceptance traceability

Express is a third fulfillment value; its fee is applied after the merchandise discount and never inherits standard delivery's free-delivery threshold.

CR requirement	Implementation and evidence
1. Service, commands and menu accept express	Service validation and pricing branch; argparse choices for quote/place; updated menu prompt. Tests in test_express.py and test_interfaces.py.
2. Always charge 1,200 cents	EXPRESS_DELIVERY_CENTS constant; unconditional express branch. High-value regular and student orders assert the fee remains.
3. Preserve merchandise discount and rounding	Shared integer formula retained. Exact $50 threshold and below-threshold merchandise tested with express; fee never counted toward eligibility or discounted.
4. Preserve pickup and standard delivery	Existing baseline tests retained; table-driven checks compare all three options, including free standard delivery.
5. Quote/order agreement and persisted snapshots	Express quote/place comparison; reopen, retrieval, history and cancellation equality checks include all original item and monetary fields.
6. Preserve validation and lifecycle	Invalid express requests reject without save; repeated cancellation preserves bytes; stock restored once; next ID increases. Other fulfillment strings remain invalid.
7. Discoverability and unchanged defaults	Both command help screens expose express; menu prompt includes express and $12.00; README and functional spec updated. Regular/pickup defaults tested.

Client examples: independently expected and observed

Request	Merchandise / discount / fee	Final total
Regular, two notebooks, express	1,000 / 0 / 1,200 cents	2,200 cents ($22.00)
Student, two hoodies, express	8,000 / 800 / 1,200 cents	8,400 cents ($84.00)
Regular, three hoodies, delivery	12,000 / 0 / 0 cents	12,000 cents ($120.00)
Regular, three hoodies, express	12,000 / 0 / 1,200 cents	13,200 cents ($132.00)

All listed examples pass. The student hoodie example also runs through a real CLI process; scripted menu placement displays $84.00 and persists express. No addresses, delivery promises, external services or real payments were added.

6. Critical evaluation of AI recommendations

This implementation and report were created with Codex assistance after the user confirmed Checkpoint A submission. The following is an AI-assisted review of recommendations, not a claim that the student independently authored or validated this critique. The full decision record is `docs/ai-review.md`.

Accepted: share quote construction

A common builder directly addresses duplicated policy. It operates on the state supplied by the caller, avoiding a second load during placement. The refactor was tested before rule corrections and retained the same failing cases. This supports the choice more concretely than a generic claim that deduplication always improves design.

Adapted: dictionary indexing and duplicate aggregation

Indexing is used only for internal inventory updates. Directly indexing raw external product IDs could turn a malformed list/dict ID into TypeError instead of required ValueError; both inputs are tested. Similarly, quantities are aggregated for stock checking while returned lines remain separate. This fixes overselling without unnecessarily changing the observable item representation.

Rejected: floating-point discounts

Using `round(subtotal * 0.1)` is shorter but does not express the required half-up rule and introduces unnecessary floating-point arithmetic. The retained integer formula returns 501 cents for the 5,005-cent fixture. A downward-rounding mutation fails, showing the test distinguishes the rule at its sensitive point. The floating-point alternative was evaluated, not committed as implementation.

Rejected: reuse the standard free-delivery condition for express

Sharing this condition would contradict the client's fixed-fee requirement. Express has its own unconditional fee branch. Tests above the standard threshold for regular and student customers detect both waived and discounted express mutations. Similar-looking fulfillment options do not imply identical policy.

Rejected: broader architecture replacement

A delivery strategy hierarchy or database would add complexity and alter the assignment's small existing application. Three fixed choices fit the current conditional, and JSON persistence meets the single-instance scope. No measurable performance or extensibility benefit was established to justify a rewrite.

Human review still required

AI recommendations were checked against source requirements and executable examples. They can still omit rubric requirements or make incomplete judgments. The student should review the actual diff, run the tests, and replace or expand this critique with their own assessment. No missing Checkpoint A content or unperformed human validation has been fabricated.

7. Reproduction and final handoff

Run the application and checks

From the cloned application directory:

```
python --version
python -m supplydesk
python -m unittest discover -s tests -v
python scripts/verify.py
git diff --check
```

The application and verification tests need no third-party packages. The PDF build helper uses ReportLab, a reporting-only dependency available in the Codex bundled runtime; it is not required to run or test the application.

```
python scripts/build_report.py
```

`scripts/verify.py` overwrites evidence files 07-09 with fresh observations; staged development logs 01-06 remain intact. It copies only application and test folders into temporary mutation directories. Every scenario uses temporary save files, preserving ordinary local application data.

Deliverables available locally

- Application changes in `supplydesk/service.py`, `cli.py` and `menu.py`.
- Original smoke tests plus pricing, validation, lifecycle, express and interface suites under `tests/` (49 test methods total).
- `docs/CR-01.md`, updated README and functional specification.
- Editable report source, AI review and testing evidence record under `docs/`.
- Recorded logs, mutation outputs and source/test hashes under `evidence/`.
- PDF review report under `output/pdf/` and a local Git development history.

Remaining submission steps

The user's explicit approval is required before pushing anything. A student-owned GitHub repository has not been created or published. The existing origin points to the instructor's starter; it must not be used as the destination for the student submission. After approval, create/use the student's own repository and publish the reviewed branch/history there, then record that actual repository URL.

The full handout must still be checked for report format, identity fields, required AI records and grading criteria. The student's Checkpoint A remains a separate prior submission. No evidence of its timestamp or submission interval was available to inspect, and none is asserted here beyond the user's confirmation.

Submit the reviewed final PDF, own repository link and testing evidence record through the final A2 D2L folder. No D2L submission or remote push was performed during this local implementation.